

Projets S5 - 17148

Mohamed Dyaé Chellaf, Julia Dunoyer

Semestre 5

Département Informatique Année 2022-2023



Sommaire

1	Introduction	3
1.1	Présentation du sujet	3
1.2	Règles du jeu	3
2	Mise en place du plateau de jeu	4
2.1	Définition des éléments principaux	4
2.2	Fonctions d'initialisation du plateau de jeu	4
2.3	Complexité de <i>world</i>	5
2.4	Visuel du jeu	5
3	Voisins et relations	6
3.1	Les fonctions neighbor(s)	6
3.2	Tests de nos fonctions neighbors	7
4	Le type abstrait ensemble	7
4.1	définition	7
4.2	Les primitives du type abstrait ensemble	7
5	Les mouvements	8
5.1	Définition des mouvements	8
5.1.1	Les mouvements simples	8
5.1.2	Les sauts simples	9
5.1.3	Les sauts multiples	9
5.2	Le choix du saut	10
6	Mises en place du jeu	11
6.1	La boucle de jeu	11
6.2	Les victoires	12
7	L'arbre des dépendances	12
8	Conclusion	14
8.1	Arborescence des fichiers	14
8.2	Notre avancement	14
8.3	Difficultés rencontrées	14

1 Introduction

1.1 Présentation du sujet

Ce projet a pour objectif la création d'une partie d'un jeu de dame. Nous utilisons pour cela, uniquement le langage C. Cet exercice a permis de mettre en application et de renforcer nos connaissances apprises en environnement de travail et en programmation impérative.

Notre projet comporte différents type de fichiers. Tout d'abord, des fichiers `.c` pour l'implémentation des fonctions dont leurs définitions sont incluses dans des fichiers `.h`. Puis, la compilation de ces fonctions se fait à l'aide d'un fichier `Makefile`. En parallèle, nous avons créé des fichiers test de façon à vérifier nos fonctions.

1.2 Règles du jeu

Les règles de ce jeu sont similaires à celles d'un jeu de dame classique. Elles sont expliquées plus précisément dans le document des consignes du projet.

Nous avons dû au cours de notre travail faire certains choix pour l'implémentation de nos fonctions et la représentation des éléments du jeu. Ceux-ci seront expliqués plus en détails dans les parties suivantes de ce document. Certains détails des règles seront aussi explicités pour mieux comprendre ces choix.

2 Mise en place du plateau de jeu

2.1 Définition des éléments principaux

Afin de mettre en place notre jeu, nous avons commencé par créer et définir les premiers éléments qui constituent le plateau.

Dans le fichier *geometry*, est défini la taille du plateau de jeu (*WORLD_SIZE*), sa hauteur (*HEIGHT*) et sa largeur (*WIDTH*).

Nous avons ensuite défini trois énumérations:

- La première, *color_t* définit les couleurs des pions du jeu.
- La seconde, *sort_t* définit les différents pions. 0 indique une absence de pion, 1 indique la présence d'un pion classique. On peut ensuite complexifier le plateau de jeu en y ajoutant des pions différents. On ajoute la tour associée dans cette structure au chiffre 2 et l'éléphant au chiffre 3. Ces deux derniers éléments ont été implémentés mais ne seront pas utilisés dans notre version de jeu.
- La dernière, *dir_t* qui définit toutes les directions possibles dans le but de se déplacer d'une case à une autre sur le plateau de jeu. De -4 jusqu'à -1 pour désigner les directions du sud-est jusqu'à l'ouest puis de 1 à 4 pour les directions de l'est jusqu'au nord-est. La valeur 0 désigne l'absence de direction. Il y a au total 9 types de directions différentes.

Les fonctions *place_to_string* et *dir_to_strind* renvoient respectivement un message décrivant la couleur et le type de pion pour l'une et la direction pour l'autre.

Puis, nous avons défini le plateau de jeu.

Pour cela, nous avons créé deux structures, *world_t* et *cpl*.

cpl comprend deux entiers, un désignant une couleur de *color_t* et un autre désignant un pion de *sort_t*. Le plateau de jeu défini par la structure *world_t* correspond à un tableau d'éléments de la structure *cpl*. Le nombre d'éléments dans le tableau est égale à *WORLD_SIZE*. Les indices dans le tableau vont alors de 0 jusqu'à *WORLD_SIZE*-1. A chaque indice du tableau on a accès à la couleur et au pion correspondant à la case sur le plateau de jeu.

2.2 Fonctions d'initialisation du plateau de jeu

Les fonctions *world_set* et *world_set_sort* prennent en paramètre respectivement un plateau de jeu, une case de ce plateau et une certaine couleur ou un certain type de pion. Elles ont pour action d'insérer des pions et des couleurs dans le plateau de jeu avec ces différentes informations.

Les fonctions *world_get* et *world_get_sort* prennent en paramètre un plateau de jeu et une case de ce plateau. Elles renvoient respectivement la couleur ou le pion, positionné sur la case

regardée.

cette partie nous a permis d'apprendre à manipuler les structures et les pointeurs. C'était quelques choses avec lesquelles nous n'étions pas très à l'aise au départ. Les manipuler à travers l'écriture de ces fonctions nous a aidé à mieux comprendre leur fonctionnement et à ainsi les utiliser efficacement.

Nous avons ensuite créé plusieurs fonctions permettant d'initialiser les éléments du plateau de jeu. La fonction *world_init* initie le plateau de jeu en considérant qu'il n'y a encore aucun pion sur le plateau. Cette fonction présente une complexité en temps linéaire.

La fonction *pos_init* prend en paramètre un plateau de jeu et place sur chaque côté ouest et est du plateau les pions des deux joueurs, d'un côté les pions noirs et de l'autre les pions blancs. Lorsque l'on place les pions, on considère qu'ils sont tous de même type et sont donc placés sans distinction. La complexité temporelle de cette fonction est proportionnelle à `WORLD_SIZE/WIDTH`.

●				○
●				○
●				○
●				○

Figure 1: *world* avec les positions des joueurs initialisées.

2.3 Complexité de *world*

Un *world* initialisé a donc une complexité spatiale et temporelle égale à `WORLD_SIZE`.

2.4 Visuel du jeu

Nous avons créé une fonction *show* permettant de représenter notre monde.

Chaque parenthèse représente une case du plateau avec (couleur du pion, type du pion).

```

(11)(00)(00)(00)(21)

(11)(00)(00)(00)(21)

(11)(00)(00)(00)(21)

(11)(00)(00)(00)(21)

```

Figure 2: Représentation d'un monde de taille 4X5 avec les pions des joueurs en position initiale

3 Voisins et relations

Nous avons ensuite défini des liens entre les cases du plateau de jeu. Chaque case peut avoir jusqu'à huit voisins, un voisin dans chacune des directions.

3.1 Les fonctions `neighbor(s)`

La fonction `get_neighbor` prend en argument un indice et une direction et renvoie le voisin correspondant dans cette direction ou `UINT_MAX` si il n'existe pas.

Pour réaliser cela, nous avons écrit une fonction comportant plusieurs tests (10 au total). Ces tests permettent de considérer les différents cas et notamment celui où la direction pointerait vers l'extérieur du plateau de jeu. Dans ce cas, il n'y a pas de voisin dans cette direction et la fonction renvoie `UINT_MAX`. Ces tests sont effiaces et répondent au problème posé. Cependant, ils augmentent la complexité temporelle de la fonction. En effet, si la direction pointe vers l'extérieur du plateau, la fonction réalise les dix tests avant de renvoyer `UINT_MAX`.

Si la fonction est appelée plusieurs fois alors la complexité temporelle est d'autant plus importante. C'est le cas de `get_neighbors`. Cette fonction renvoie tous les voisins associés à la case d'indice passé en paramètre.

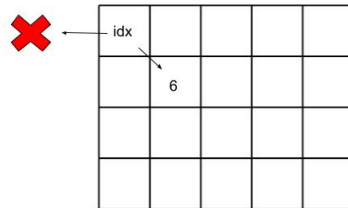


Figure 3: La case d'indice `idx` ne possède pas de voisin à gauche mais en possède un en bas à droite d'indice 6, un autre en bas et un troisième à droite.

3.2 Tests de nos fonctions neighbors

Nous avons créé une fonction test afin de vérifier le cas où le voisin recherché n'existerait pas. Nous avons testé aussi la fonction *get_neighbors* pour s'assurer que l'ensemble des voisins trouvés par la fonction étaient bien les bons.

4 Le type abstrait ensemble

4.1 définition

Il a été ensuite nécessaire de créer un nouveau type abstrait permettant de contenir un ensemble d'éléments. Nous avons fait le choix de créer une nouvelle structure appelée *ensemble*.

```
struct ensemble {  
    int e[2*WORLD_SIZE];  
    int MAX;  
};
```

Le premier élément de cette structure est un tableau d'entier. Nous avons fait le choix d'initialiser ce tableau à une taille égale à deux fois `WORLD_SIZE`. Nous avons considéré la taille de ce tableau suffisante pour regrouper les données nécessaires.

En effet, cet ensemble aura pour but notamment de regrouper les indices des cases correspondant aux mouvements possibles d'une pièce sur le plateau de jeu. Ce nombre sera donc inférieur à la taille du tableau initialisé.

Le deuxième élément de cette structure est le nombre d'éléments utilisés dans le tableau. Il est utile de pouvoir accéder rapidement au nombre d'éléments dans le tableau, sans faire appel à une fonction extérieure.

4.2 Les primitives du type abstrait ensemble

Ce type abstrait présente plusieurs primitives. Voici-ci celles que nous avons créées et ensuite utilisées dans notre code.

Tout d'abord, la fonction *choose_random* prend en paramètre un ensemble et renvoie un élément choisi aléatoirement parmi cet ensemble.

```
int choose_random(struct ensemble pieces)  
{  
    int r = rand()%pieces.MAX;  
    return pieces.e[r];  
}
```

La fonction fait appel à *rand()*. Celle-ci présente un défaut qui est que si la fonction est appelée plusieurs fois à la suite, elle renverra toujours la même suite de valeurs.

Nous avons créé une seconde fonction qui réalise la même chose que celle précédente mais ne renvoie pas l'élément "0" s'il fait parti de l'ensemble. Cette fonction permettra plus tard de choisir une direction parmi un ensemble contenant toutes les directions possibles, sans choisir la direction "absence de direction".

Si l'ensemble contient que des zéros elle renvoie alors "0".

La complexité temporelle de cette seconde fonction est plus importante que la première. En effet, la première présente une complexité égale à celle de la fonction *rand* tandis que la deuxième a une complexité en $O(\text{pieces.MAX})$.

Enfin, nous avons implémenter une troisième fonction *choose_random_belonging_to* qui fait appel à *choose_random*. Cette fonction réalise la même tâche que celle-ci. Nous avons choisis de la créer pour que le code soit plus clair et compréhensible.

5 Les mouvements

5.1 Définition des mouvements

5.1.1 Les mouvements simples

Le mouvement le plus simple est le mouvement simple. Il correspond à un déplacement sur une case voisine si celle-ci existe et est disponible.

Nous avons créé une première fonction : *int verifie_simple_move(struct world_t* b, unsigned int idx, enum color_t c, enum dir_t d)* qui va vérifier si la case voisine de la case *idx* qui contient un pion de la couleur *c* dans la direction *d* possède un pion. Si la case en question est vide, la fonction renvoie 1, sinon 0.

La fonction *simple_move(struct world_t* b, unsigned int idx, enum color_t c, enum dir_t d)* réalise le mouvement du pion de la couleur *c* sur la case voisine dans la direction *d* si cela est possible.

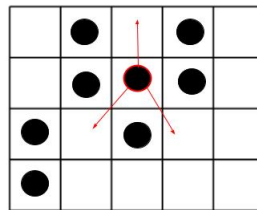


Figure 4: Les flèches représentent les trois mouvements simples possibles pour le pion sélectionné

5.1.2 Les sauts simples

Le saut simple consiste à faire sauter un pion au dessus d'un autre, depuis sa position vers une case libre (ne contenant pas déjà un pion). De façon similaire au mouvement simple, nous avons créé une fonction *verifie_simple_jump* qui prend en paramètre les même arguments que la fonction *verifie_simple_move* et renvoie 1 si le saut simple est possible. pour cela, elle réalise un premier test qui consiste à verifier si il y a bien un pion de couleur adverse sur la case voisine dans la direction demandée. Si cette condition est vérifiée alors, un nouveau test est réalisé pour verifier si la case voisine du voisin dans la direction demandée est libre. Si cela est le cas alors la fonction renvoie 1. Si un des test est faux alors la fonction renvoie 0. La fonction *simple_move* fait bouger le pion si les conditions du saut simple sont réunies.

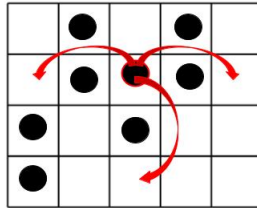


Figure 5: Les flèches représentent les trois sauts simples possibles pour le pion sélectionné

5.1.3 Les sauts multiples

Les sauts multiples correspondent à un enchaînement de sauts simple pouvant être réalisés à chaque fois dans des directions différentes. Cette caractéristique rend l'implémentation de la fonction plus compliquée que celles précédentes.

Nous avons d'abord créé la fonction *shuffle* qui prend en argument un tableau d'entiers et sa taille et renvoie ce même tableau en ayant mélangé ses éléments.

La fonction *verifie_multiple_jump* prend en paramètre les mêmes arguments que les fonctions précédentes. Un premier test est réalisé pour vérifier qu'un saut simple est possible. Si ce n'est pas le cas la fonction renvoie 0. Sinon, une nouvelle direction va être sélectionnée dans un tableau contenant l'ensemble des directions et dont les éléments ont été mélangés à l'aide de la fonction *shuffle*. Puis on test à nouveau si un saut simple est possible dans cette direction depuis la position obtenue au saut simple précédent. Si ce n'est pas possible alors la fonction renvoie 0, il n'y a pas eu de saut multiple. Sinon la fonction continue à réaliser ce processus jusqu'à ce qu'un saut simple ne soit plus possible. A chaque fois une nouvelle direction est sélectionnée aléatoirement.

Nous avons fait le choix de faire intervenir l'aléatoire. Cependant, cette fonction présente une limite qui est qu'un saut multiple peut ne pas être détecté si la direction vers ce saut n'est pas sélectionnée.

La fonction *saut_multiple* doit bouger le pion pour le déplacer jusqu'à la dernière position obtenue pour un saut multiple. Nous avons eu un problème avec l'implémentation de cette fonction. Elle existe dans notre version actuelle du projet mais ne réalise pas cette tâche attendue.

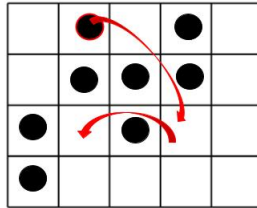


Figure 6: Les flèches représentent les successions de sauts du saut multiple possible pour le pion sélectionné

5.2 Le choix du saut

Nous avons ensuite créé une fonction permettant de recueillir les indices de tous les mouvements possibles. Il s'agit de la fonction *get_possible_moves*. Elle prend en argument un monde, un indice sur le plateau, un ensemble et la couleur des pions d'un des joueurs.

Elle est divisée en trois parties pour chacun des différents types de mouvements. Trois tableaux sont créés pour chacun des types de mouvements et ont pour utilité de contenir l'ensemble des directions dans lesquels le mouvement est possible. Ces tableaux peuvent avoir jusqu'à huit éléments. Pour les mouvements simples et les sauts simples, cela représente l'ensemble des mouvements possible dans toutes les directions. Pour les sauts multiples, il pourrait y en avoir plus. Cependant, nous avons choisis de garder seulement une unique possibilité de saut multiple pour chacun des sauts partant de la position de départ vers les 8 directions possibles. Une fois les trois tableaux remplis, une direction est sélectionnée au hasard pour chacun d'eux grâce à la fonction *random_choice_not_zero*. Et la direction est stockée dans l'ensemble donné au départ en paramètre.

Nous avons fait le choix de privilégier dans l'ordre un saut multiple, puis un saut simple et enfin un mouvement simple. La fonction renvoie alors l'indice de l'élément dans le tableau des ensembles correspondant au saut sélectionné.

Enfin, une dernière fonction *move_piece* réalise le mouvement correspondant à celui-choisi

par la fonction précédente.

Ce mouvement est réalisé à partir de la direction dans lequel on veut l'initier. Pour un mouvement simple ou un saut simple il s'agit du même mouvement que celui trouvé par la fonction précédente. Cependant, pour un saut multiple dont la fonction fait appel à l'aléatoire, il est possible que le mouvement ne soit pas le même ou même que finalement le mouvement sélectionné ne puisse aboutir. Le premier saut sera dans la direction sélectionnée mais les suivants seront dans une direction quelconque.

6 Mises en place du jeu

6.1 La boucle de jeu

Une fois les fonctions des mouvements implémentés, il n'a pas été compliqué de mettre en place la boucle de jeu.

Nous avons choisis deux ensembles distincts contenant chacun l'indice des pions du joueur considéré. La fonction *set_pieces* crée ces ensembles. Chaque joueur va jouer séparément jusqu'à ce qu'un des joueurs gagne ou que le nombre de tour maximal soit atteint.

Avant de commencer d'implémenter la boucle, on a créé une nouvelle fonction qui va nous être utile pour passer d'un joueur à son adversaire, et qui s'appelle *next_player*.

Ensuite, avant de rentrer dans le corps de la boucle, la fonction commence déjà par initialiser un monde grâce à la fonction *world_init*, puis met les pions des deux joueurs en positions de départ grâce à la fonction *set_init*, remplit ensuite la structure ensemble (*Struct ensemble X_pieces*) de chaque joueur par les indices occupés par ce joueur, et qui sont les indices initiaux pour l'instant.

La fonction fait ensuite appel à *get_random_player* pour obtenir le joueur qui va commencer la partie, et initialise un compteur *turn* pour faire le compte des tours, et affiche ainsi la table de jeu initiale.

LA PARTIE EST DESORMAIS EN JEU.

Ensuite, la fonction fait appel à quasiment toutes les autres fonctions définies auparavant.

Elle commence par appeler dans cet ordre, les fonctions *choose_random_piece_belonging_to* pour obtenir une pièce aléatoire du joueur *X*, puis *get_possible_moves* pour obtenir les différents mouvements possibles pour la pièce choisie, ensuite elle fait appel à *move_piece* pour effectuer le mouvement choisi par la fonction précédente.

Pour la structure de la boucle, elle est en première partie divisée en deux parties indépendantes, chacune concernant une couleur, et à l'intérieur de chaque partie, le mouvement du pion choisi de la couleur concernée est effectué (ou pas) et la structure du joueur est modifiée pour enregistrer les nouvelles positions après que le mouvement soit effectué.

En seconde partie, c'est le test des victoires, les fonctions des victoires sont alors appelées

pour décider si la partie est terminée par la victoire d'un joueur ou que la partie continue en implémentant 1 à *turn*, en affichant à la fin la table de jeu instantannée.

Bien sur, cette boucle a une fin même si personne ne gagne, en effet, au moment où *turn* arrive à la valeur *MAX.TURNS*, la partie prend fin en affichant la table de jeu finale et que le match est nul.

6.2 Les victoires

Tout d'abord, il y a deux types de victoires : la victoire simple et la victoire complexe. Comme leurs noms l'indiquent, la première est simple et consiste à ce qu'un pion du joueur *X* arrive à se placer dans l'une des cases de départ de l'autre joueur, pour que le joueur *X* gagne, tandis que la deuxième est plutôt complexe et consiste à ce que toutes les cases de départ de l'autre joueur soient occupées par les pions du joueur *X* pour que ce dernier gagne.

Pour l'implémentation du code, on a choisi de séparer les fonctions du contrôle de la victoire entre les joueurs BLACK et WHITE, en effet, on a implémenté 2 fonctions concernant la victoire simple : une qui concerne le joueur BLACK et l'autre concernant le joueur WHITE, et 2 fonctions concernant celle complexe: une qui concerne le joueur BLACK et l'autre concernant le joueur WHITE.

- *Victoire Simple*: Pour une couleur *X* du joueur, cette fonction prend en paramètre un monde *w*, et contient une boucle qui traverse les cases initiales de l'adversaire en testant si l'une d'entre elles est occupée par un pion du joueur *X*, et au si elle en trouve un, elle retourne vrai (1), et dans le cas où elle traverse toutes les cases sans rien trouver, elle retourne faux (0).
- *Victoire Complexe*: Pour cette fonction, et étant donné un joueur de couleur *X*, la fonction décrivant ce type de victoire est implémentée d'une autre manière. Elle prend en paramètre un monde et initialise un compteur à 0. Elle contient ensuite une boucle qui traverse toutes les cases de départ de l'adversaire et ajoute 1 au compteur chaque fois qu'elle rencontre un pion du joueur *X*. A la fin de la boucle, la fonction teste si la valeur finale du compteur est égale au nombre de lignes (HEIGHT). Dans ce cas, elle renvoie vrai (1). Si la valeur est inférieure à ce nombre, elle renvoie faux (0).

Les fonctions testant les victoires sont appelées à la fin de chaque tour de la partie pour décider si l'un des joueurs a gagné ou que la partie continue.

Ces deux fonctions ont une complexité temporelle à peu près égale au nombre des lignes du monde.

7 L'arbre des dépendances

Cet arbre est assez complexe et rend ainsi compte de la dépendance de nos fichiers entre eux. Cela représente un réel handicap dans le cas où nous serons amenés à modifier un des fichiers

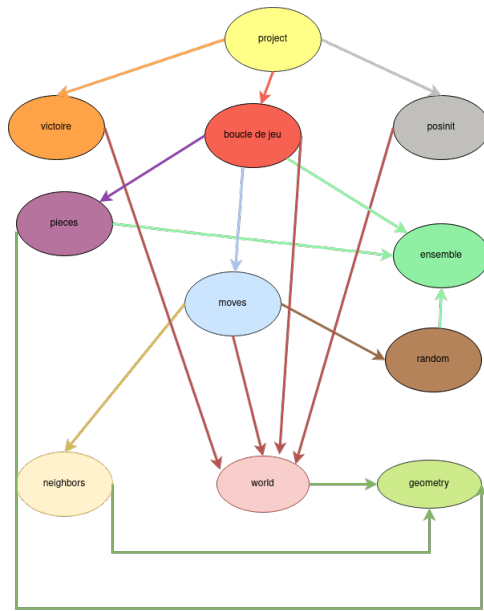


Figure 7: L'arbre des dépendances

et alors l'ensemble des fichiers qui en dépendent seraient obligés d'être modifiés eux aussi.

8 Conclusion

8.1 Arborescence des fichiers

Au départ, on ne savait quasiment rien sur les détails et l'utilité des fichiers `.h` et `.c` et les liens entre eux, et du coup, se fut l'occasion de découvrir le mode de fonctionnement d'un code en entier sous forme de plusieurs fichiers, les uns dépendants des autres, et une fois qu'on a compris le mode de fonctionnement, l'établissement des algorithmes et l'enchaînement des étapes sont devenus plus facile.

8.2 Notre avancement

Nous sommes arrivés à l'achèvement 0. Certaines parties de notre code seraient encore à clarifier. Nous pouvons aussi chercher à diminuer la complexité de nos fonctions. Chaque élément de ce projet étant au départ nouveau pour nous, nous avons passé un temps important à comprendre l'utilisation des outils autour du code et un peu moins au début à coder nos fonctions. Cela justifie le fait que nous sommes arrivés uniquement à cet Achievement. Cependant, il était très important de consacrer ce temps pour construire de bonnes bases et pouvoir ensuite lors de nos futurs projets partir avec un bagage plus solide.

De plus, vers la fin du projet (les deux dernières semaines, nous avons rencontrés des échecs du code et des erreurs de ségmentation compliquées alors qu'on faisait l'Achievement 1 en parallèle, du coup on a décidé d'abandonner ce dernier et de se concentrer sur l'Achievement 0 vu son importance pour le projet, et on y est resté jusqu'à résoudre toutes les erreurs de code et de ségmentation associées, et aussi les petites erreurs du Makefile.

8.3 Difficultés rencontrées

Ce projet fut très enrichissant. En effet, la programmation en C, était pour nous deux quelque chose de nouveau. Le fait de coder chaque semaine nous a fait progresser très rapidement. La démarche ludique du projet a aussi été un moteur dans notre démarche d'apprentissage.

Nous avons rencontré plusieurs difficultés durant la construction de ce projet.

Tout d'abord, nous ne connaissions pas Git avant de commencer. Nous avons donc du consacrer le temps nécessaire au début pour bien maîtriser cet outil. Ce fut aussi l'occasion d'apprendre à mieux organiser l'ensemble de nos fichiers. Une rigueur de la gestion des fichiers est nécessaire et cela est d'autant plus important lorsque nous travaillons en groupe. Nous avons appris aussi à mettre en place un Makefile et toute la structure de code impliquant les `.h` et les `.c` des fichiers, chose qui présentait pour nous un grand obstacle au début du projet, et désormais, nous sommes conscients de l'importance de ces connaissances pour la réalisation de projets.

Nous avons compris en construisant le Makefile, comment la compilation des fichiers était réalisée.

Aussi toute la démarche de réalisation de tests pour s'assurer de la fiabilité de nos fonctions était quelque chose de nouveau pour nous.

Nous avons énormément progressé en codage en C. Nous maîtrisons beaucoup mieux les pointeurs, les enums et les structures.

Plus nous avons avancé dans le projet, plus nous nous sommes sentis à l'aise à coder. Il a été aussi très intéressant de prendre nos propres décisions pour le code et non pas répondre simplement à une consigne.

Avec du recul nous sommes conscients que le choix que nous avons fait de faire intervenir le hasard dans la sélection des directions de la fonction *multiple_jump* ne permet pas de réaliser des tests satisfaisant pour vérifier la fiabilité de notre fonction.

Et enfin, nous avons aussi appris à coder en groupe sous SSH, et de savoir gérer la globalité des fichiers mis en jeu sans avoir de conflit, et aussi répartir les tâches entre nous entre le codage et le Makefile.